Complete finite validation program

Human Compression Project

This entire program is validation, not a uniform theorem premise. It constructs two actual semidirect products, checks all marked subset intersections and every triple in the flag-transitivity criterion. The complete raw file is included below; no routine is omitted from its cost boundary.

```
# Complete small-model validation. No uniform theorem depends on these cases.
# Standard generated parabolics, all subset intersections, and the full
# three-parabolic flag-transitivity criterion are checked, not only orders.
Require := function(test,msg) if not test then Error(msg); fi; end;
FullGeometry := function(G,marks,name,expected,orders)
 local n,sets,pars,J,K,H,P,Q,R,lhs,rhs,k,i,j,rows;
 n:=Length(marks); Require(Size(G)=expected,Concatenation(name," order"));
 Require(Group(marks)=G,Concatenation(name," generation"));
 Require(ForAll(marks,x->Order(x)=2),Concatenation(name," involutions"));
 for i in [1..n] do for j in [i+1..n] do
  if j<=n then Require(Order(marks[i]*marks[j])=orders[i][j],"pair order"); fi;
 od; od;
 sets:=Combinations([1..n]);
 pars:=List(sets,S->Subgroup(G,List(Difference([1..n],S),i->marks[i])));
 for J in sets do for K in sets do
  P:=pars[Position(sets,J)]; Q:=pars[Position(sets,K)];
  Require(Intersection(P,Q)=pars[Position(sets,Union(J,K))],"IP");
 od; od;
 # Product subsets are precomputed once. Direct elements, no quotient assumptions.
 rows:=List(pars,P->List(pars,Q->Set(Concatenation(List(Elements(P),x->List(Elements(Q),y->x*y))))
 ↪ ));
 for i in [1..Length(sets)] do for j in [1..Length(sets)] do
  for k in [1..Length(sets)] do
   lhs:=Intersection(rows[i][k],rows[j][k]);
   rhs:=rows[Position(sets,Union(sets[i],sets[j]))][k];
   Require(lhs=rhs,"full coset flag-transitivity");
  od;
 od; od;
 Print("MODEL ",name," order=",Size(G)," IP=",Length(sets)^2," FT=",Length(sets)^3," PASS\n");
end;

MakeModel:=function(fixed)
 local F,fg,rel,i,j,W,iso,A,ag,outer,B,bg,aut0,aut1,aut,phi,G,eA,eB,marks,n,ord;
 n:=3; if fixed then n:=4; fi;
 F:=FreeGroup(n);fg:=GeneratorsOfGroup(F);rel:=List(fg,x->x^2);
 for i in [1..n] do for j in [i+1..n] do
  if j<=n then
   if fixed and j=4 then Add(rel,(fg[i]*fg[j])^3);
   else Add(rel,(fg[i]*fg[j])^2);fi;
  fi;
 od;od;
 W:=F/rel;iso:=IsomorphismPermGroup(W);A:=Image(iso);
 ag:=List(GeneratorsOfGroup(W),x->Image(iso,x));
 B:=Group((1,2),(2,3));bg:=[(1,2),(2,3)];
 aut0:=GroupHomomorphismByImages(A,A,ag,Concatenation([ag[2],ag[1],ag[3]],ag{[4..n]}));
 aut1:=GroupHomomorphismByImages(A,A,ag,Concatenation([ag[1],ag[3],ag[2]],ag{[4..n]}));
 Require(aut0<>fail and aut1<>fail,"Coxeter graph action");
 aut:=Group(aut0,aut1);phi:=GroupHomomorphismByImages(B,aut,bg,[aut0,aut1]);
 Require(phi<>fail and Size(Image(phi))=6,"B action");
```

```
 G:=SemidirectProduct(B,phi,A);eB:=Embedding(G,1);eA:=Embedding(G,2);
 marks:=[Image(eA,ag[1]),Image(eB,bg[1]),Image(eB,bg[2])];
 if fixed then marks:=Concatenation([Image(eA,ag[4])],marks);fi;
 ord:=List([1..Length(marks)],i->List([1..Length(marks)],j->2));
 if fixed then ord[1][2]:=3;ord[2][3]:=4;ord[3][4]:=3;
  FullGeometry(G,marks,"fixed-vertex D4 by S3",1152,ord);
 else ord[1][2]:=4;ord[2][3]:=3;
  FullGeometry(G,marks,"three commuting vertices by S3",48,ord);
 fi;
end;
MakeModel(false);MakeModel(true);
# A negative control for order exactness: a trivial projection cannot force doubling.
Require(Order((1,2,3))=3 and Order((1,2,3)^2)=3,"negative model changed");
Print("DIAGONAL_MODELS_COMPLETE\n");
QUIT;
```